

MANAGING PROCESSES

TASKS

1. Open your Linux terminal and use the ps command to display all currently running processes for your user. Save the output to a file named my_processes.txt.

Display Running Processes

Step 1 Display running processes

ps

Or display all processes

ps -ef

Step 2 Save output to a file

ps -ef > my_processes.txt

Step 3 Verify

cat my_processes.txt

```
[root@localhost ~]# cat my_processes.txt
UID          PID    PPID  C STIME TTY          TIME CMD
root          1        0  0  21:09 ?        00:00:06 /usr/lib/systemd/systemd --system --des
erialize 21 rhgb
root          2        0  0  21:09 ?        00:00:00 [kthreadd]
root          3        2  0  21:09 ?        00:00:00 [pool_workqueue_]
root          4        2  0  21:09 ?        00:00:00 [kworker/R-rcu_gp]
root          5        2  0  21:09 ?        00:00:00 [kworker/R-sync_wq]
root          6        2  0  21:09 ?        00:00:00 [kworker/R-slub_flushwq]
root          7        2  0  21:09 ?        00:00:00 [kworker/R-netns]
root          9        2  0  21:09 ?        00:00:00 [kworker/0:0H-events_highpri]
root         11        2  0  21:09 ?        00:00:00 [kworker/R-mm_percpu_wq]
```

**2. Start the Spotify client (or any music player app available on your Linux system), then use the top command to find its process ID (PID) and note it down.

Hint: Look for the process name in the top list or use pgrep if needed.**

Start a Process and Find PID

Since Spotify is usually not installed on CentOS, we'll use the sleep command as a long-running process.

Step 1 Start a background process

sleep 300 &

```
[root@localhost ~]# sleep 300 &
[1] 51809
[root@localhost ~]#
```

Step 2 Find the PID

ps -ef | grep sleep

OR

pgrep sleep

OR

top

Press **q** to exit top.

```
[1] 51809
[root@localhost ~]# ps -ef | grep sleep
root      51809      3318  0 23:30 pts/0    00:00:00 sleep 300
root      51837      3318  0 23:32 pts/0    00:00:00 grep --color=auto sleep
[root@localhost ~]# pgrep sleep
51809
[root@localhost ~]# top

top - 23:34:15 up 2:24, 1 user, load average: 0.00, 0.01, 0.04
Tasks: 231 total, 2 running, 229 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.7 sy, 0.0 ni, 97.7 id, 0.3 wa, 1.0 hi, 0.0 si, 0.0 st
MiB Mem : 732.2 total, 66.5 free, 511.8 used, 275.5 buff/cache
MiB Swap: 2048.0 total, 1449.3 free, 598.7 used. 220.4 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
  785 root        20   0 384504   3164  2616  S   0.3   0.4   0:23.57 vmtoolsd
 1004 root        20   0 484532   1708   916  S   0.3   0.2   0:02.82 tuned
 1050 apache      20   0 983144    348   276  S   0.3   0.0   0:03.23 httpd
 2783 root        20   0 3496152 94544 34412  S   0.3  12.6   1:47.25 gnome-shell
 3050 root        20   0 539776    7784  4388  S   0.3   1.0   0:23.37 vmtoolsd
 51849 root        20   0 225936    4260  3404  R   0.3   0.6   0:00.19 top
```

3. Use the kill command to terminate the process you found in the previous step. Verify that the process has stopped by running ps or top again.

Kill the Process

Step 1 Kill the process

Replace 12345 with your actual PID.

kill 12345

Step 2 Verify

`ps -ef | grep sleep`

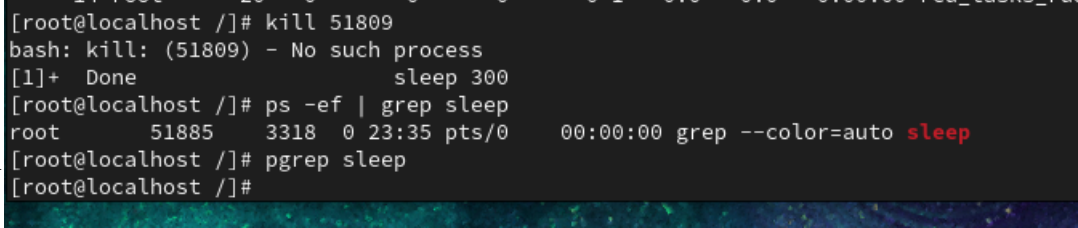
Expected:

Only the grep sleep line should appear.

Or

`pgrep sleep`

No output means the process has stopped.



```
[root@localhost ~]# kill 51809
bash: kill: (51809) - No such process
[1]+  Done                  sleep 300
[root@localhost ~]# ps -ef | grep sleep
root      51885      3318  0 23:35 pts/0    00:00:00 grep --color=auto sleep
[root@localhost ~]# pgrep sleep
[root@localhost ~]#
```

4.Run

a

long-

running command like `ping google.com` in the background using `&`. Then, bring it to the foreground and stop it using `Ctrl+Z` and the `fg` command.
Hint: Use the `jobs` command to see background jobs.

Background and Foreground Jobs

Step 1 Run ping in background

`ping google.com &`

Step 2 Check background jobs

`jobs`

Step 3 Bring job to

```
[root@localhost ~]# ping google.com &
[1] 53013
[root@localhost ~]# PING google.com (142.251.126.113) 56(84) bytes of data.
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=1 ttl=128 time=56
8 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=2 ttl=128 time=35
0 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=3 ttl=128 time=17
4 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=4 ttl=128 time=94
.1 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=5 ttl=128 time=10
9 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=6 ttl=128 time=13
4 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=7 ttl=128 time=15
2 ms
64 bytes from cn-in-f113.1e100.net (142.251.126.113): icmp_seq=8 ttl=128 time=12
1 ms
```

foreground

Fg

The ping starts running in the terminal.

Step 4 Suspend it

Press:

ctrl + z

You'll see:

Stopped

Step 5 View jobs

jobs

Step 6 Bring it back

fg

Step 7 Stop it completely

Press

ctrl + c

5. Suppose you accidentally started a heavy process that is slowing down your system (e.g., a big file copy or a simulation). Use the nice and renice commands to lower its priority and explain in one line how this helps system performance.

nice and renice

Step 1 Start a process with low priority

nice -n 10 sleep 300 &

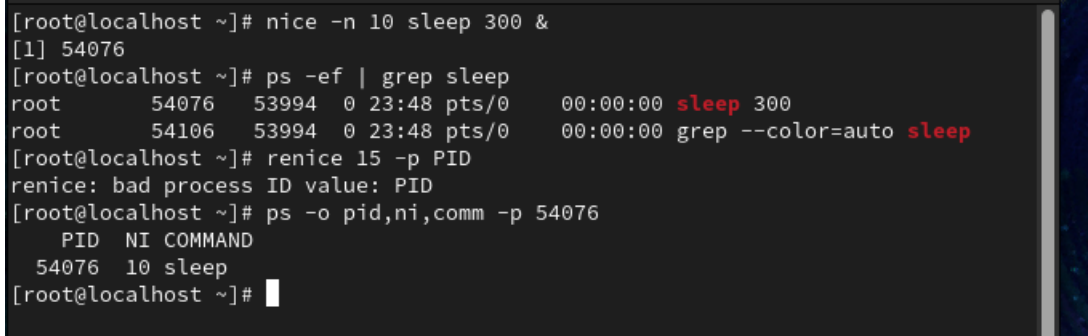
Step 2 Find PID

ps -ef | grep sleep

Step 3 Change priority

Replace PID with your PID.

renice 15 -p PID



```
[root@localhost ~]# nice -n 10 sleep 300 &
[1] 54076
[root@localhost ~]# ps -ef | grep sleep
root      54076    53994  0 23:48 pts/0    00:00:00 sleep 300
root      54106    53994  0 23:48 pts/0    00:00:00 grep --color=auto sleep
[root@localhost ~]# renice 15 -p PID
renice: bad process ID value: PID
[root@localhost ~]# ps -o pid,ni,comm -p 54076
  PID  NI  COMMAND
  54076  10  sleep
[root@localhost ~]#
```

Explanation

Question: How does nice and renice help?

Answer:

nice starts a process with lower CPU priority, and renice changes the priority of a running process. This allows important processes to get more CPU time and improves overall system performance.